EXP 1	**Develop a predictive text system using N-Gram and HMM (Hidden Markov Models) to predict the next word based on previous context.** **Dataset: Brown Corpus or Wikipedia dataset.**

AIM:

To develop a **predictive text system** that predicts the **next word based on previous context** using **N-Gram language models** and **Hidden Markov Models (HMM)**. The system is trained on a **text corpus such as the Brown Corpus or Wikipedia dataset** to learn word patterns and probabilities for generating accurate next-word predictions.

PROCEDURE:

1. **Dataset Loading:** Load the Brown Corpus using NLTK. Extract raw sentences and POS-tagged sentences using the Universal tagset.
2. **Preprocessing:** Convert text to lowercase. Remove non-alphabetic characters using regex. Tokenize using NLTK's word_tokenize.
3. **N-Gram Construction:** Generate Unigrams, Bigrams, and Trigrams from tokens. Compute frequency counts using Counter.
4. **N-Gram Prediction:** For single-word input use Bigram with Laplace smoothing. For multi-word input use Trigram, falling back to Bigram then Unigram.
5. **HMM Training:** Train a Hidden Markov Model using NLTK's HiddenMarkovModelTrainer on the first 5,000 POS-tagged sentences.
6. **HMM Prediction:** Tag input using HMM, get the last POS tag, predict the next POS tag via transition probabilities, then pick a word using emission probabilities.
7. **User Interface:** An interactive menu lets the user choose between N-Gram and HMM modes, enter text, and receive next-word predictions.

CODE:

```
# ==========================================
# COMBINED NLP MODEL
# 1. N-Gram Language Model
# 2. Hidden Markov Model
# Dataset: Brown Corpus
# ==========================================

import nltk
import re
import random
import warnings
from nltk.corpus import brown
from nltk.tokenize import word_tokenize
from nltk.util import ngrams
from nltk.tag import hmm
from collections import Counter

warnings.filterwarnings("ignore")

# ------------------------------------------
# DOWNLOAD DATASETS
# ------------------------------------------
nltk.download('brown')
nltk.download('punkt')
nltk.download('universal_tagset')

# ==========================================
# LOAD DATASET
# ==========================================
sentences = brown.sents()
```

```
tagged_sentences = list(brown.tagged_sents(tagset='universal'))
text = " ".join([" ".join(sent) for sent in sentences])

# ==========================================
# PREPROCESSING
# ==========================================
text = text.lower()
text = re.sub(r'[^a-z\s]', '', text)
tokens = word_tokenize(text)
tokens = [t for t in tokens if t.strip() != ""]
print("Total Tokens:", len(tokens))

# ==========================================
# N-GRAM MODEL
# ==========================================
unigrams = list(ngrams(tokens, 1))
bigrams  = list(ngrams(tokens, 2))
trigrams = list(ngrams(tokens, 3))

unigram_freq = Counter(unigrams)
bigram_freq  = Counter(bigrams)
trigram_freq = Counter(trigrams)
vocab_size   = len(set(tokens))
print("Vocabulary Size:", vocab_size)

def predict_unigram():
    return unigram_freq.most_common(1)[0][0][0]

def predict_bigram(word):
    candidates = {
        w2: count
        for (w1, w2), count in bigram_freq.items()
        if w1 == word
    }
    if not candidates:
        return None
    probs = {
        w: (count + 1) / (unigram_freq[(word,)] + vocab_size)
        for w, count in candidates.items()
    }
    return max(probs, key=probs.get)

def predict_trigram(word1, word2):
    candidates = {
        w3: count
        for (w1, w2, w3), count in trigram_freq.items()
        if w1 == word1 and w2 == word2
    }
    if not candidates:
        return None
    return max(candidates, key=candidates.get)

# ==========================================
# HMM MODEL
# ==========================================
trainer   = hmm.HiddenMarkovModelTrainer()
hmm_model = trainer.train(tagged_sentences[:5000])
print("HMM Model Trained Successfully")

def predict_next_word_hmm(sentence):
    words    = sentence.lower().split()
    tagged   = hmm_model.tag(words)
    last_tag = tagged[-1][1]
    next_tag = max(
        hmm_model._states,
        key=lambda tag: hmm_model._transitions[last_tag].prob(tag)
    )
    word_probs = {
        word: hmm_model._outputs[next_tag].prob(word)
        for word in hmm_model._outputs[next_tag].samples()
```

```
    }
    top_words = sorted(
        word_probs.items(), key=lambda x: x[1], reverse=True
    )[:5]
    return random.choice(top_words)[0]

# ==========================================
# USER MENU
# ==========================================
while True:
    print("\n========= NLP NEXT WORD PREDICTION =========")
    print("1. N-Gram Prediction")
    print("2. HMM Prediction")
    print("3. Exit")
    choice = input("Enter your choice: ")

    if choice == "1":
        user_input = input("Enter a word / phrase: ").lower()
        user_input = re.sub(r'[^a-z\s]', '', user_input)
        words = user_input.split()
        if len(words) == 1:
            prediction = predict_bigram(words[0])
            if prediction:
                print("Predicted next word:", prediction)
            else:
                print("Fallback Prediction:", predict_unigram())
        elif len(words) >= 2:
            prediction = predict_trigram(words[-2], words[-1])
            if prediction:
                print("Predicted next word:", prediction)
            else:
                prediction = predict_bigram(words[-1])
                if prediction:
                    print("Predicted next word:", prediction)
                else:
                    print("Fallback Prediction:", predict_unigram())

    elif choice == "2":
        sentence   = input("Enter a sentence: ")
        prediction = predict_next_word_hmm(sentence)
        print("Predicted next word:", prediction)

    elif choice == "3":
        print("Program Stopped")
        break

    else:
        print("Invalid choice")
```

Output:

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Package universal_tagset is already up-to-date!

Total Tokens: 1005406
Vocabulary Size: 46273
HMM Model Trained Successfully

========= NLP NEXT WORD PREDICTION =========
1. N-Gram Prediction
2. HMM Prediction
3. Exit
Enter your choice: 1
Enter a word / phrase: he said
Predicted next word: he
```

```
========= NLP NEXT WORD PREDICTION =========
1. N-Gram Prediction
2. HMM Prediction
3. Exit
Enter your choice: 2
Enter a sentence: he said
Predicted next word: said

========= NLP NEXT WORD PREDICTION =========
1. N-Gram Prediction
2. HMM Prediction
3. Exit
Enter your choice: 3
Program Stopped
```

Result:

The predictive text system was successfully implemented using **N-Gram** and **Hidden Markov Model** techniques. The trained model was able to analyze the context of previous words and predict the most probable next word based on learned probability distributions from the dataset.

The N-Gram model captured local word dependencies effectively, while the **HMM model** helped model hidden linguistic structures and sequence probabilities. The system demonstrated the capability to generate context-aware next word predictions, showing how statistical language models can be used in applications such as autocomplete, search engines, and smart keyboards.